

PROJECT REPORT

Comment Category Prediction Challenge

A Macro-F1-Optimized, Metric-Aware Ensemble Pipeline for Imbalanced Comment Classification

COMMENT_CATEGORY_PREDICTION | DATA CONSULTANT SUBMISSION

Abstract

Categorizing user-generated comments at platform scale is a practical, high-stakes text classification problem, complicated here by severe class imbalance and a scoring metric that is not disclosed in closed form. This project builds a supervised classification pipeline that predicts the final moderation category assigned to a comment, using its raw text, engagement signal (upvotes and downvotes), a timestamp, and a set of system-detected topic flags. Following exploratory analysis of the target distribution and per-class text and engagement patterns, the raw comment text was vectorized with TF-IDF and combined with eighteen engineered features covering text shape, temporal position, engagement, and thread activity. Five classifiers — Logistic Regression, Complement Naive Bayes, SGD, LightGBM, and Random Forest — were trained and compared individually, then combined via a coordinate-ascent weighted blend that optimizes validation macro-F1 directly. A first attempt at ensembling, weighted by accuracy, scored worse on the leaderboard than a single model despite higher local accuracy; comparing submission scores against local metrics across iterations revealed the competition's undisclosed custom metric tracks macro-F1, not accuracy — expected given the roughly 58/8/32/3 percent class split. Re-optimizing every modeling decision around macro-F1 recovered and then exceeded the original baseline, reaching a leaderboard score of 0.79640.

1. Introduction

1.1 Problem Statement

Automated content moderation systems assign each user comment to one of several handling categories, and predicting that assignment from a comment's observable features is a natural benchmark for disciplined text-classification workflow: text vectorization, tabular feature engineering, model comparison, and metric-aware model selection. The task is framed as a 4-class classification problem: given a comment's text, engagement counts, timestamp, and system-detected topic flags, predict the variable **label**. The target distribution is heavily imbalanced, which materially affects which evaluation metric is appropriate and which models generalize well.

1.2 Objectives

- Profile the dataset to characterize target imbalance, feature types, and class-conditional patterns.
- Engineer text-shape, temporal, engagement, and thread-activity features from the raw columns.
- Vectorize comment text with TF-IDF and combine it with the engineered numeric features.

- Train and compare multiple classifiers of different inductive biases (linear, naïve Bayes, boosted trees, bagged trees).
- Identify the true evaluation metric empirically by comparing local validation scores against leaderboard feedback.
- Select a final blend based on validation performance under the identified metric, not raw accuracy.
- Generate a correctly formatted submission file.

2. Dataset Description

The dataset consists of individual comment records from an online discussion platform, split into a training set of 198,000 rows and a test set of 102,000 rows, with 15 and 14 columns respectively. Table 1 summarizes the documented schema.

Column	Description
comment	Raw text content of the comment
created_date	Date and time the comment was posted
post_id	Identifier linking the comment to its parent discussion thread
emoticon_1 / 2 / 3	Indicators for three internal emoticon groups
upvote / downvote	Positive / negative reaction counts
if_1 / if_2	Internal features hidden by the platform
race / religion / gender / disability	System-detected references to each identity/topic category
label (Target)	Final handling category assigned by the platform — 4 classes

Table 1. Dataset schema and feature descriptions.

A preliminary diagnostic pass — row counts, dtypes, missingness, and class balance — confirmed a clean load with the four label values distributed at approximately 57.7%, 8.0%, 31.5%, and 2.8% of the training set. This imbalance, more than any single feature, is the dominant characteristic of the dataset and drives most of the modeling decisions described in Section 3.

3. Methodology

3.1 Exploratory Data Analysis

EDA covered missing-value patterns, the target class distribution, comment length and word count by class, engagement (total votes) by class, and the prevalence of each system-detected topic flag by class. Comment length and word count showed visible differences across classes, as did total engagement, supporting their inclusion as engineered features alongside the raw text. Topic-flag prevalence varied unevenly by class, consistent with these flags carrying real, if partial, predictive signal rather than being noise.

3.2 Feature Engineering

Eighteen features were engineered from the raw columns and combined with TF-IDF text vectors, summarized in Table 2.

Feature group	Description
Text shape	Character length, word count, average word length, unique-word ratio, uppercase/digit ratios, exclamation/qu
Temporal	Hour, day-of-week, and month extracted from created_date
Engagement	Total votes (upvote + downvote), vote ratio, log-transformed total votes
Thread activity	Comment count per post_id, computed across train + test to avoid target leakage
Text vector	Word-level TF-IDF, 1–2 grams, 10,000 features, English stopwords removed

Table 2. Engineered features used in modeling.

The TF-IDF matrix was combined with the standardized numeric features into a single sparse matrix for the linear models and LightGBM; Complement Naive Bayes used the raw non-negative TF-IDF matrix alone, and Random Forest was trained on the numeric/engineered features only, deliberately excluded from the text signal to maximize its diversity within the eventual ensemble.

3.3 Model Training and Validation

Five classifiers were trained and compared: Logistic Regression (multinomial, $C = 1.5$, class-balanced) on the combined sparse matrix; Complement Naive Bayes ($\alpha = 0.3$) on the TF-IDF matrix; SGD with log-loss (class-balanced) on the combined sparse matrix; LightGBM (150 estimators, 31 leaves, class-balanced) trained directly on the sparse matrix without dimensionality reduction; and Random Forest (200 estimators, max depth 12, balanced-subsample) on the numeric features alone. Models were evaluated on a stratified 88/12 train/validation split, scored on both accuracy and macro-averaged F1.

3.4 Metric Discovery

The competition page identifies only a “Custom Metric” without a closed-form definition. An initial accuracy-optimized ensemble scored 0.74040 on the leaderboard — lower than a single Logistic Regression model's 0.77489 — despite the ensemble's higher validation accuracy (0.898 vs. approximately 0.90 for Logistic Regression alone). Tabulating validation macro-F1 alongside leaderboard scores across submissions showed the two tracking almost exactly ($0.7824 \rightarrow 0.77489$; $0.7426 \rightarrow 0.74040$), while accuracy showed no such correspondence. This was taken as strong evidence that the custom metric is macro-F1 or functionally equivalent to it, and every subsequent modeling decision — class weighting, model selection, and ensemble weighting — was re-optimized around macro-F1 accordingly.

3.5 Ensembling Strategy

Rather than a fixed-order greedy blend, a coordinate-ascent search was used: starting from equal weights across all five models, each model's weight is swept over a fine grid with every other model's weight held fixed, keeping any change that improves validation macro-F1, across several full passes. A model that never improves the blend converges to weight zero and is excluded from the final refit, which both guarantees the ensemble can only match or beat the best single model on the validation set and avoids wasted compute refitting unused models on the full training data.

4. Results and Model Performance

The five candidate models achieved the following validation performance on the held-out 12% split.

Model	Val. Accuracy	Val. Macro-F1
Logistic Regression	0.8915	0.7806
Complement Naive Bayes	0.6922	0.6121
SGD (log-loss)	0.8808	0.7591
LightGBM	0.8965	0.7832
Ensemble (coordinate ascent)	0.9008	0.7981

Table 3. Validation accuracy and macro-F1 by model, stratified 88/12 split.

LightGBM was the strongest individual model on macro-F1, narrowly ahead of Logistic Regression; Complement Naive Bayes was comfortably the weakest individually but, like Random Forest, contributed diversity that measurably improved the blend despite its low standalone score — the same pattern by which a low-variance, moderately-weaker model can still raise an ensemble's ceiling. The coordinate-ascent blend improved macro-F1 over the best single model (LightGBM, 0.7832) by roughly 1.5 points, reaching 0.7981 on validation and 0.79640 on the leaderboard — a gap of under half a percentage point, consistent with the macro-F1 metric theory established in Section 3.4.

4.1 Score Progression

Iteration	Strategy	Leaderboard Score
V1	Logistic Regression only	0.77489
V2	5-model blend, weighted by accuracy	0.74040
V3 / V4	5-model blend, selected/weighted by macro-F1	0.79640

Table 4. Leaderboard score across major iterations of the pipeline.

The regression between V1 and V2 was not a modeling failure in isolation — it was the direct, predictable consequence of optimizing for the wrong metric. Correcting the objective, rather than adding further model complexity, was what recovered and ultimately exceeded the original baseline.

5. Conclusion

This project demonstrates a complete classification pipeline for an imbalanced, metric-ambiguous competition: the dataset was profiled for class imbalance and per-class patterns; text and tabular features were combined into a unified representation; five diverse classifiers were trained and compared; and a coordinate-ascent search was used to blend them against the metric that was empirically determined, rather than assumed, to govern the leaderboard. The result — a validation macro-F1 of 0.7981 and a leaderboard score of 0.79640 — represents a nearly 3-point improvement over the original single-model baseline, achieved primarily by correcting the optimization target rather than by adding model complexity.

6. Future Work

- Grouped (by post_id) train/validation split, to rule out thread-level leakage inflating validation scores relative to the true leaderboard.
- Character-level TF-IDF or subword/embedding representations alongside the current word-level TF-IDF.
- Stacking with a trained meta-learner in place of the coordinate-ascent weight search.
- Per-class threshold or decision-boundary tuning, given a metric that weights all four classes equally regardless of their size.
- Broader hyperparameter search (grid or Bayesian) for LightGBM and Random Forest.

Appendix — Technical Stack

The project is implemented in Python within a Jupyter notebook, using the following core libraries:

- pandas & numpy — data loading, cleaning, and feature computation
- matplotlib & seaborn — exploratory and diagnostic visualizations
- scikit-learn — TF-IDF vectorization, scaling, linear models, Naive Bayes, Random Forest, cross-validation, and metrics
- LightGBM — gradient-boosted tree classifier

Key output artifacts generated by the pipeline:

- submission.csv — final label predictions in the required submission format
- Class-distribution, comment-length, engagement, and flag-prevalence charts by category
- Model-comparison bar chart (accuracy vs. macro-F1) and confusion matrix for the top individual model